Jeff Preshing Blog

Brian

April 17, 2026

Contents

1 2011

1.1 Nov 18 - Lock Contention

A mutex (**Mu**tual **ex**clusion) prevents multiple threads from accessing the same piece of data at the same time. This is document with a single "key" that only one thread can own at a time. Consider a situation with two threads, A and B:

1. Thread A calls `mutex.lock()`. Since it is already unlocked, A can access the critical section.
2. Thread A enters the critical section and modifies shared data.
3. Thread B calls `mutex.lock()`. Since thread A already holds the mutex (the key), thread B is blocked, and must wait.
4. Thread A exists the critical section, and calls `mutex.unlock()`.
5. Thread B can now access the critical data, and it calls `mutex.lock()`.

Note that the lock itself must be atomic, i.e. it must be done in one step. This is to ensure that two threads dont try to lock/unlock the mutex at the same time. Common locking instructions include test-and-set and compare-and-swap (CAS). While mutexes are widely used since they are easy to implement and suffice for 90% of applications, there is a notion that mutexes tend to be "slow", especially compared to other data-race prevention patterns, and that lock-free patterns should almost always be used. This is false; the lock itself is extremely fast. There are two main "bottlenecks" in locks: the overhead and contention. The misconception comes in the fact that the performance hit is only in the contention; after all, the overhead cannot be more than a few nanoseconds since the lock itself must be atomic. The issue comes in the fact that while thread A is accessing the critical section, thread B must wait and essentially "do nothing": it is blocked. This is known as lock contention, and is the key issue that lock-free structures try and prevent (although this dramatically increases the complexity for implementation)

1.2 Nov 24 - Lightweight Mutex

The Windows Software Development Kit (SDK) provides developers with two mutex implementations, called **mutex** and **critical section** (pretty bad names imo - why name the lock itself the critical section??). The critical section a lightweight mutex, and is optimized for the case where there are no other threads competing for the lock. consider the following snippets:

```
// Windows Mutex
HANDLE mutex = CreateMutex(NULL, FALSE, NULL);
for (int i = 0; i < 1000000; ++i) {
  WaitForSingleObject(mutex, INFINITE);
  ReleaseMutex(mutex);
}
CloseHandle(mutex);
```

```
// Windows Critical Section
CRITICAL_SECTION critSec;
InitializeCriticalSection(&critSec);
for (int i = 0; i < 1000000; i++){
    EnterCriticalSection(&critSec);
    LeaveCriticalSection(&critSec);
}
DeleteCriticalSection(&critSec);
```

After benchmarking, the critical section implementation is around 25x faster than the mutex implementation. This is because the mutex implementation invokes syscalls per every lock/unlock, adding a lot of overhead. The critical section is purely in userspace, at the tradeoff that it cannot be shared between multiple processes.

2 2012

2.1 May 15 - Memory Reordering

Earlier the idea of lock-free design patterns was briefly mentioned; a way to have multiple threads access the same data with minimal contention. But how is this actually done? One very important caveat that must be considered is the idea of correct memory ordering. In (basically all) modern programming languages, the compiler and cpu will perform optimizations and reorder instructions to improve performance. Consider the following situation, where we have two integers X and Y, initially set to 0, and two processors running in parallel:

Processor 1	Processor 2
mov [X], 1 *;store to X*	mov[Y], 1 *;store to Y*
mov r1, [Y] *;load from Y*	mov r2, [X] *;load from X*

Note that r1 and r2 are placeholders for registers, such as eax.
Now, you may expect this to be quite intuitive: no matter which processor writes 1 to memory first, since the other processor reads the value we should end up with either r1 = 1, r2 = 1, or both. However, according to Intel's official x86 specification, this is actually undefined behaviour; it is perfectly legal for r1 = r2 = 0 at the end of this example. This is because on most processor families, the processor is allowed to reorder the memory interactions according to certain rules, so long as it never changes the outcome of a **single-threaded** program. Thus, in this case, it may reorder the above in the following:

Processor 1	Processor 2
mov r1, [Y]	
	mov r2, [X]
mov [X], 1	
	mov [Y], 1

In the following code snippet, we can actually see this in action. we will spawn two worker threads that will indefinitely repeat the process described above. For the first worker thread:

```
sem_t beginSema1;
sem_t endSema;

int X, Y;
int r1, r2;

void *thread1Func(void *param)
{
    MersenneTwister random(1);                  // Initialize random
       number generator
    for (;;)                                    // Loop indefinitely
    {
        sem_wait(&beginSema1);                  // Wait for signal from
           main thread
        while (random.integer() % 8 != 0) {}    // Add a short, random
           delay

        // ----- THE TRANSACTION! -----
        X = 1;
        asm volatile("" ::: "memory");          // Prevent compiler
           reordering
        r1 = Y;

        sem_post(&endSema);                     // Notify transaction
           complete
    }
    return NULL;  // Never returns
};
```

Note that POSIX semaphore is used to synchronzie the threads at the beginning of the loop, and X, Y, r1, and r2 are all global variables. And below is the main thread:

```
int main()
{
    // Initialize the semaphores
    sem_init(&beginSema1, 0, 0);
    sem_init(&beginSema2, 0, 0);
    sem_init(&endSema, 0, 0);

    // Spawn the threads
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, thread1Func, NULL);
    pthread_create(&thread2, NULL, thread2Func, NULL);

    // Repeat the experiment ad infinitum
    int detected = 0;
    for (int iterations = 1; ; iterations++)
    {
        // Reset X and Y
        X = 0;
        Y = 0;
        // Signal both threads
        sem_post(&beginSema1);
        sem_post(&beginSema2);
        // Wait for both threads
        sem_wait(&endSema);
        sem_wait(&endSema);
```

```
26          // Check if there was a simultaneous reorder
27          if (r1 == 0 && r2 == 0)
28          {
29              detected++;
30              printf("%d reorders detected after %d iterations\n",
                    detected, iterations);
31          }
32      }
33      return 0;  // Never returns
34  }
```

Running this we can observe that there is a memory reordering that causes both r1 and r2 to be 0 once every 6600 iterations.

2.2 June 12 - Introduction to Lock-Free Programming

So what exactly is lock-free programming? In it's core, its just a property of multithreaded code that accesses some shared memory in a way that the threads never block each other. Due to the sheer complexity of lock-free code, large codebases are almost never entirely lock-free; instead, there are a specific set of operations that are deemed to be lock-free. Below are a few common techniques for lock free programming:

2.2.1 Atomic Read-Modify-Write (RMW) Operations

As discussed previously, an atomic operation is one that is indivisible; it occurs in the shortest possible timestep from the procsesor's point of view. RMW operations go further on this, allowing you to perform more complex operations. A classic example in C++ is `std::atomic::fetch_add`. When this is called, the system will perform (conceptually) three steps as a single atomic operation:

1. Read: It reads the current value of the atomic variable.
2. Modify: It adds some value to the variable.
3. Write: It writes the newely calculated value of the variable.

On x86 architectures, `fetch_add` maps to the `LOCK XADD` (exchange and add) instruction. Note that with C++ atomics, the standard does not enforce that the implementation for the type will always be lock free; thus it is a good idea to include the line `static_assert(std::atomic<type>::is_always_lock_free)` to ensure that your code will work on the architecture it is running on.

2.2.2 Compare-And-Swap (CAS) loops

Perhaps the most common RMW operation is CAS. This essentially tells the system to check a specific memory location, and if it is exactly equal to some expected value, update it and return true. else return false. The gcc compiler has a builtin `__atomic_compare_exchange`, which can be used as follows:

```
void LockFreeQueue::push(Node* newHead) {
  for (;;) {
    // copy a shared head to local
    Node* old_head = m_head;
    // do some work, not visible on other threads
    newHead->next = old_head;
    // perform cas to ensure that the shared variable has not changed
       while performing work; else repeat
    if (__atomic_compare_exchange(&m_head, newHead, oldHead)) return ;
  }
}
```

2.2.3 Sequential Consistency

Sequential Consistency will enforce all threads to agree on the order in which memory operations are to occur, in a way that is consistent with the order of operations of the program source code. This causes issues with memory reordering to be impossible. On C++, operations on `std::atomic` are, by default, sequencially consistent (however this can be changed to weaker orderings for better performance). With sequencial consistency, if thread A sees value X change before value Y, thread B and all other threads are guaranteed to see this exact same order. This ensures that all threads share a single ground truth, which helps reduce the complexity of writing lock-free code. This comes with the tradeoff, however, in that threads

2.3 September 13 - Acquire-Release Semantics

Generally, in lock-free programming, there are two ways in which two threads can access shared memory; they can either compete against each other for it or pass information from one to another. Acquire-release semantics are extremely important for the latter, espcecially in cases where sequencial consistency is not guaranteed or when performance is critical. Essentially, acquire-release creates "barriers" that synchronize threads to ensure that they dont access memory in an invalid state. It can be seperated into two distinct categories:

1. Acquire Semantics: A property that pertains only to operations that read from shared memory. Ensures that no memory operations that occur after the acquire get reordered to occur before.
2. Release Semantics: A property that pertains only to operations that write to shared memory. Ensures that no memory operations that occur before the release get reordered to occur after.

More clearly,

- Thread A (Producer)
  1. Prepare `Data = 42`
  2. Release: `Flag = true` - Ensures that everything preceeding stays above the flag.
- Thread B (Consumer)
  1. Acquire: assert `Flag == True` - Ensures that everything proceeding stays below the flag
  2. Read `Data = 42`

Concretely, in C++:

```cpp
// This will pass on basically all modern architectures, but still good to check
static_assert(std::atomic<bool>::is_always_lock_free);
std::atomic<bool> ready{false};
int data = 0; //NOT ATOMIC!

void producer() {
  //prepare:
  data = 42;
  //Release: "data = 42" will stay Above this line
  ready.store(true, std::memory_order_release);
}

void consumer() {
  //Acquire: Everyhing below this line stays below this line. First check that the ready flag is true, and spin wait if not
  while (!ready.load(std::memory_order_acquire)) {
    _mm_pause(); // x86 spin wait
  }
  assert(data == 42); //this assertion is guaranteed to pass
  std::cout << "Data␣received:␣" << data << "\n";
}
```

The important thing to notice here is that each acquire and release are one-way fences; for exapmle with acquire, it only enforces memory reordering for operations following the acquire, but doesnt care about anything before, and vice-versa with release.